A Hybrid Clock System Related to STeC Language

Yixiang Chen
MoE Engineering Research Center for
Software/Hardware Co-design Technology and Application
Shanghai Key Lab for Trustworthy Computing
East China Normal University
Shanghai 200062, China

Yuanri Zhang
Software Engineering Institute
East China Normal University
Shanghai 200062, China
University of Nice Sophia Antipolis
Nice, France

***Abstract*—Cyber-Physical Systems (CPS) is a new trend of real-time systems in the area of distributed embedded systems or networked agent systems. The first author introduced a specification language for real-time system, called as spatial-temporal consistency language (Shortly, STeC) in 2010. In this paper, the authors introduce a novel clock system, called as hybrid clock, to specify both logical and chronometric time aspect of real time system. Some operations on hybrid clocks and relations between hybrid clocks are introduced. A satisfaction relation between a hybrid clock and a STeC design of real time system specified in term with STeC language is defined. Some properties and CPS case studies are given in this paper.**

***Index Terms*—Cyber-Physical Systems, Real-Time Systems, Hybrid clock, STeC Specification Language.**

I. Introduction

Cyber-Physical Systems (CPS) is a new trend of real-time systems in the area of distributed embedded system. Intelligent transportation network is a classic example of distributed embedded systems. In intelligent transportation network, embedded entities, such as trains, aircrafts and cars, possess such capabilities as extensive interaction, computation, communication and control. In particular, they have the capability of interaction with physical environments they locate. For example, when trains, airplanes and cars arrive some places or, then they should know their tasks, e.g., communicating with their neighbors or information center, or doing special tasks.

The first author introduces in [4] a specification language for real-time systems, called as **STeC**, stressing the spatial-temporal consistence of real-time systems. This spatial-temporal consistence means that an agent arrives at the required location at the required time and finishes the required tasks within the required times. For example, the train G147 arrives the station Xuzhou at 17:28 and stays there for 8 minutes and then leaves that station at 17:36. That train G147 will go to Nanjing station with 75 minutes and arrives at Nanjing station at 18:51. STeC language can be used to specify the CPS real-time systems. The details of STeC language will be restated in the section II.

Here is the schedule of Train G147 from Beijing to Shanghai in the Jinghu Gaotie(high-speed train) line.

Beijing ($a\,14:30$, $d\,14:36$) $\xrightarrow[131km]{34m}$ TianJin ($a\,15:10$, $d\,15:12$) $\xrightarrow[288km]{63m}$ Jinan ($a\,16:15$, $d\,16:17$) $\xrightarrow[206km]{50m}$ Zaozhuang ($a\,17:07$, $d\,17:09$) $\xrightarrow[63km]{19m}$ Xuzhou ($a\,17:28$, $d\,17:36$) $\xrightarrow[390km]{75m}$ Nanjing ($a\,18:51$, $d\,18:53$) $\xrightarrow[149km]{51m}$ Suzhou ($a\,19:42$, $d\,19:43$) $\xrightarrow[75km]{23m}$ Shanghai ($a\,20:06$, $d\,20:11$)

Clock system specifies the system time. Normally, there are two types of clock time: logical time and physical/chronometric time.

Logical time has proved its benefits in several domains. It was first introduced by Lamport to represent the execution of distributed systems [8]. It has then been extended and used in distributed systems to check the communication and causality path correctness [5]. Logical time has also been intensively used in synchronous languages [2] for its multiform nature. In the synchronous domain it has proved to be adaptable to any level of description, from very flexible causal time descriptions to very precise scheduling descriptions [3]. MARTE [11] Time model deals with both discrete and dense time. In MARTE, a clock gives access to a time structure. A clock can be either chronometric or logical. The former is related to physical time while the latter is not.

Andr$\acute{e}$ and Mallet in [1] and later Mallet in [10] define a clock to be a 5-tuple $<\mathcal{I}, \prec, \mathcal{D}, \lambda, u>$. Recently, He Jifeng in [6] defines that a clock c is an infinite increasing sequence of non-negative reals, which is indeed an increasing monotonic function from the natural number $\mathcal{N}$ to the real number $\mathcal{R}$. Each value $c(i)$ is an instant, called as the i^{th} instant of clock c. Therefore a discrete-time clock indeed is the same as He's clock of being an infinite increasing sequence of non-negative reals.

The contribution of this paper is to introduce a novel clock system, called as hybrid clock, based on Mallet's clock and He's clock, to specify the time aspect, logical and chronometric time, of real time system. Some operations on hybrid clocks and relations between hybrid clocks are introduced. A satisfaction relation between a hybrid clock and a design of real time system specified in term with STeC language is defined. Some properties and case studies for sensors and trains as well as power network are given in this paper.

II. STeC Language

STeC language is a specification language for real-time systems with the constraints of the spatial-temporal consis-

978-1-4799-5843-6/14 $31.00 © 2014 IEEE
DOI 10.1109/SERE-C.2014.39

tency introduced by Chen in [4]. Its formal semantics and tool set have been developed in [4], [7], [9], [12]. Here is the extended version of STeC in [4], [12].

$$A \quad ::= \quad \mathsf{Send}^{G\rightharpoonup G'}_{(l,t)}(m,\delta) \mid \mathsf{Get}^{G\leftharpoonup G'}_{(l,t)}(m,\delta)$$

$$B \quad ::= \quad \mathsf{Act}^{G}_{(l,t)}(l',\delta) \mid \mathsf{Stat}^{G}_{(l,t)}(\delta)$$

$$C \quad ::= \quad \mathsf{Sens}^{G}_{(l,t)}(m,\delta) \mid \mathsf{Comp}^{G}_{(l,t)}(m,m',\delta)$$

$$P \quad ::= \quad A \mid B \mid C \mid P;P \mid P \parallel P \mid$$

$$P \trianglerighteq (\mathsf{Get}^{G\leftharpoonup G'}_{(l,t)}(m,\delta) \rightarrow P)$$

STeC looks like an extension of process systems CSP and CCS. A stands for these two atomic communications $\mathsf{Send}^{G\rightharpoonup G'}_{(l,t)}(m,\delta)$ and$\mathsf{Get}^{G\leftharpoonup G'}_{(l,t)}(m,\delta)$. $\mathsf{Send}^{G\rightharpoonup G'}_{(l,t)}(m)$ means that agent G will send a message m to agent G' at location l and time t. Whilst the command $\mathsf{Get}^{G\leftharpoonup G'}_{(l,t)}(m)$ means that at location l and at time t, agent G receives a message m sent by G'. Both takes the duration δ. B stands for the action and status of agents. $\mathsf{Act}^{G}_{(l,t)}(l',\delta)$ means that agent G, at the location l and time t, takes its action Act with the duration δ and arrives at the location l'. $\mathsf{Stat}^{G}_{(l,t)}(\delta)$ means that the agent G will keep its status Stat with the duration δ time at the location l and time t. C stands for these two processes of sensing and computing. $\mathsf{Sens}^{G}_{(l,t)}(m,\delta)$ means that agent G executes the process Sens at the location l and time t and outputs message m taking the δ time. $\mathsf{Comp}^{G}_{(l,t)}(m,m',\delta)$ means that agent G executes the computing process at the location l and t with the input m and the output m' within the δ time. The remains are the composition of processes: sequence ;, parallel $\parallel$ and the interrupt of message $P \trianglerighteq (\mathsf{Get}^{G\leftharpoonup G'}_{(l,t)}(m,\delta) \rightarrow Q)$. We require that agent G has gotten the message m at the location l and time t. So, the duration δ in the interrupt claus does not need to add up.

Here is some examples to specifies real-time systems using STeC.

The first example is of sensor with the capability of computing in addition of sensing and communication as well as the status of Idle.

$$\mathsf{Idle}^{N}_{(l,t)}(60s); \mathsf{Sens}^{N}_{(l,t+60s)}(m,1s); \mathsf{Comp}^{N}_{(l,t+61s)}(m,3s);$$
$$\mathsf{Send}^{N\rightharpoonup NC}_{(l,t+64s)}(x,2s); \mathsf{Idle}_{(l,t+66s)}(10s); \mathsf{Get}^{NC\leftharpoonup N}_{(l,t+76s)}(y,2s)$$

The second example is of Power Network with the notation: GZ for Guangzhou, YN for Yunnan, Cen for the control center and Contr for the action of control, Plnt for the plant and user for the user.

$$(\mathsf{Sens}^{GZ}_{(l_1,t)}(m_1,3ms) \| \mathsf{Sens}^{YN}_{(l_2,t)}(m_2,3ms));$$
$$(\mathsf{Send}^{GZ\rightharpoonup Cen}_{(l_1,t+3ms)}(m_1,2ms) \| \mathsf{Send}^{YN\rightharpoonup Cen}_{(l_2,t+3ms)}(m_2,2ms));$$
$$(\mathsf{Get}^{Cen\leftharpoonup GZ}_{(l_3,t+5ms)}(m_1,2ms) \| \mathsf{Get}^{Cen\leftharpoonup YN}_{(l_3,t+5ms)}(m_2,2ms));$$
$$\mathsf{Comp}^{Cen}_{(l_3,t+7ms)}((m_1,m_2),5ms);$$
$$(\mathsf{Send}^{Cen\rightharpoonup GZ}_{(l_3,t+12ms)}(x,2ms) \| \mathsf{Send}^{Cen\rightharpoonup YN}_{(l_3,t+12ms)}(y,2ms));$$
$$(\mathsf{Get}^{GZ\leftharpoonup Cen}_{(l_1,t+14ms)}(x,2ms) \| \mathsf{Get}^{YN\leftharpoonup Cen}_{(l_2,t+14ms)}(y,2ms));$$
$$(\mathsf{Contr}^{GZ}_{(l_1,t+16ms)}(User,5ms) \| \mathsf{Contr}^{YN}_{(l_2,t+16ms)}(Plnt,6ms))$$

The third example is the interaction between train and station when train wants to enter the station.

When entering into station for stop, a train needs to communicate with the station so that it can safely stop at platform. Along the track, there are such locations as Lappr, Lpstop and Lstop. When arriving at location Lappr, a train sends message $Stop$ to the station and then continues to run forward to the station. When the train gets the message ok at location Lpstop and goes directly into the platform with the action Stop and stops at location Lstop. Train has two actions: Run and Stop. The station can be thought as an intelligent agent. The station has two action Wk and Arg as well as one status Arged. If the station stays at state Arged then it sends message ok to train and can permit the train entering into platform and stoping at location Lstop.

The train agent can be specified using STeC as follows.

$$\mathsf{Run}_{(*,*)}(\infty); (\mathsf{Send}^{\rightharpoonup STA}_{(\mathsf{Lappr},t)}(Stop,1s) \parallel \mathsf{Run}_{(\mathsf{Lappr},t)}(Lpstop,5m))$$
$$; (\mathsf{Get}^{\leftharpoonup STA}_{(\mathsf{Lpstop},t+5m)}(ok,1s) \rightarrow \mathsf{Stop}_{(\mathsf{Lpstop},t+5m)}(Lstop,3m))$$

The station agent can be specified using STeC below.

$$\mathsf{Wk}_{(*,*)}(\infty) \triangleright (\mathsf{Get}^{\leftharpoonup TR}_{(*,t+1s)}(Stop,1s) \rightarrow \mathsf{Arg}_{(*,t+1s))}(Arged,4m));$$
$$(\mathsf{Send}^{\rightharpoonup TR}_{(\mathsf{Arged},t+4m1s)}(ok,1s) \parallel \mathsf{Arged}_{(\mathsf{Arged},t+4m1s)}(8m))$$

We use STeC to specify the schedule of Gaotie G147 as follows. This specification is quite simple. But, it is enough as a background for us to introduce the hybrid clock system which this paper focuses on.

$\mathsf{Tstop}_{(\mathsf{Beijing},14:30)}(6m)$; $\mathsf{Run}_{(\mathsf{Beijing},14:36)}(\mathsf{Tianjin},34m)$;
$\mathsf{TStop}_{(\mathsf{Tianjin},15:10)}(2m)$; $\mathsf{Run}_{(\mathsf{Tianjin},15:12)}(\mathsf{Jinan},63m)$;
$\mathsf{Tstop}_{(\mathsf{Jinan},16:15)}(2m)$; $\mathsf{Run}_{(\mathsf{Jinan};16:17)}(\mathsf{Zaozhuang},50m)$;
$\mathsf{Tstop}_{(\mathsf{Zaozhuang},17:07)}(2m)$; $\mathsf{Run}_{(\mathsf{Zaozhuang},17:09)}(\mathsf{Xuzhou},19m)$;
$\mathsf{Tstop}_{(\mathsf{Xuzhou},17:28)}(8m)$; $\mathsf{Run}_{(\mathsf{Xuzhou},17:36)}(\mathsf{Nanjing},75m)$;
$\mathsf{Tstop}_{(\mathsf{Nanjing},18:51)}(2m)$; $\mathsf{Run}_{(\mathsf{Nanjing},18:53)}(\mathsf{Suzhou},51m)$;
$\mathsf{Tstop}_{(\mathsf{Suzhou},19:42)}(1m)$; $\mathsf{Run}_{(\mathsf{Suzhou},19:43)}(\mathsf{Shanghai},23m)$;
$\mathsf{Tstop}_{(\mathsf{Shanghai},20:06)}(5m)$.

III. Hybrid Clock System

Based on André and Mallet's clock [1], [10] and He's clock [6], we introduce the notion of hybrid clock to describe both logical and chronometric time.

A. Basic Notions

André and Mallet in [1] and later Mallet in [10] define a clock to be a 5-tuple $<\mathcal{I}, \prec, \mathcal{D}, \lambda, u>$, where $\mathcal{I}$ is a set of instants (possibly infinite). $\prec$ is a total, irreflexive, and transitive binary relation (a quasi-order) on $\mathcal{I}$, named strict precedence, $\mathcal{D}$ is a set of labels, $\lambda : \mathcal{I} \rightarrow \mathcal{D}$ is a labeling function, u is a symbol, standing for a unit. But, they only considers pure clock, i.e., the ordered set $<\mathcal{I}, \prec>$ and not the labels. They define a clock to be discrete-time it this has a discrete set of instants $\mathcal{I}$. A discrete-time clock C can be indexed by natural numbers.

He Jifeng in [6] defines a clock as an infinite increasing sequence of non-negative reals. Due to the sequence, He's clock can be indexed by natural numbers. These non-negative reals state the physical time. He's clock is in the abstract level.

We will introduce a kind of clock, called as hybrid clock to merge with the logical time and chronometric time through merging of André-Mallet clock and He clock in this subsection.

Definition 3.1: (Hybrid Clock) Hybrid clock is a 5-tuple $<\mathcal{I}, D, \lambda, , \beta, u>$, where

- $\mathcal{I}$ is a set of instants (possibly infinite);
- D is an index set which is a poset with the partial order $<$ is a partial order over D;
- $\lambda : \mathcal{I} \rightarrow D$ is a labeling function, which is a bijection;
- $\beta : \mathcal{I} \rightarrow \mathbb{R}^+$ is a chronometric function, where $\mathbb{R}^+$ is the set of non-positive real numbers, whose elements stand for the chronometric time;
- u is a symbol, standing for a unit of chronometric time.

Remarks: (1) The labeling function λ gives the logical clock showing that the instant $I \in \mathcal{I}$ is the $\lambda(I)^{\text{th}}$ instant.

(2) The labeling function λ may reduce a transitive partial order on the instant set $\mathcal{I}$ defining as to be:

for any pair $I, J \in \mathcal{I}$, $I < J$ if and only if $\lambda(I) < \lambda(J)$. The resulting poset $(\mathcal{I}, <)$ is isomorphic to the index poset $(D, <)$. So, we omit the index set D if no confusion appear. We define the notion of direct successor and direct predecessor of instants. For $I, J \in \mathcal{I}$, if $I < J$ and there exist no instants $I' \in \mathcal{I}$ such that neither $I < I'$ nor $I' < J$ hold, then we call I as the direct predecessor of I and J as the direct successor of I, denoted by $I \lhd J$.

(3) The chronometric function β records the physical time. For instant I, the $\beta(I) \in \mathbb{R}^+$ is the time of instant I's occurrence. We need to require the chronometric function β is preserving the order $<$ on $\mathcal{I}$, i.e., for all $I, J \in \mathcal{I}$, if $I < J$ then $\beta(I) < \beta(J)$.

(4) u is a unit of chronometric time. We can choose the different unit time u for the different scale, e.g, u can be second scale, or minute scale and hour scale.

(5) Each instant I is associated with a logical time $\lambda(I)$ and a chronometric time $\beta(I)$. So, we have a pair function $(\lambda, \beta) : \mathcal{I} \rightarrow (\mathcal{D} \times \mathbb{R}^+)$.

(6) We use the non-negative real numbers to denote the chronometric time, since chronometric time can be represented with the time unit. For example, the time 13:40pm can be represented as the real number 49200 with the time unit of the second. However, if we choose the time unit as the minute, then the same time 13:40pm is the real number of 820.

Example: Based on the schedule of Gaotie G147, we can get these two clocks **Clock**.a and **Clock**.d with recording the schedule of arriving at and departing from stations respectively.

Clock.a(G147)=$<\mathcal{I}, D, \lambda, \beta, u>$, where

- $\mathcal{I} = \{(Beijing, 14:30), (Tianjing, 15:10), (Jinan, 16:15), (Zaozhuang, 17:07), (Xuzhou, 17:28), (Nanjing, 18:51), (Suzhou, 19:42), (Shanghai, 20:06)\}$;
- $D = \{1 < 2 < 3 < 4 < 5 < 6 < 7 < 8\}$;
- $\lambda((Beijing, 14:30)) = 1, \cdots, \lambda((Zaozhuang, 17:07)) = 4, \cdots, \lambda((Shanghai, 20:06)) = 8$;
- u is the second unit, then the chronometric time will be represented as seconds, e.g., 14:30=42200.
- $\beta((Beijing, 14:30)) = 42200, \beta((Tianjing, 15:10)) = 54600, \cdots, \beta((Shanghai, 20:06)) = 72360$.

Remark: If we choose the unit of u as the minute, then the chronometric function β changes as:

- $\beta((Beijing, 14:30)) = 870, \beta((Tianjing, 15:10)) = 910, \cdots, \beta((Shanghai, 20:06)) = 1206$.

Similarly, we define the departing clock **Clock**.d for train G147. But, we only use the instant set to represent this hybrid clock.

Clock.d(G147)=$\{(Beijing, 14:36), (Tianjin, 15:12), (Jinan, 16:17), (Zaozhuang, 17:09), (Xuzhou, 17:36), (Nanjing, 18:53), (Suzhou, 19:43), (Shanghai, 20:11)\}$

B. Healthy Clock

With a following of He's work in [6], we give the following notions.

Definition 3.2: For a given clock C, the low and high rates $\triangle(C)$ and $\nabla(C)$ are defined by

$$\triangle(C) = \mathbf{inf}\{\beta(J) - \beta(I) \mid I, J \in \mathcal{I} \wedge I \lhd J\}$$

$$\nabla(C) = \mathbf{sup}\{\beta(J) - \beta(I) \mid I, J \in \mathcal{I} \wedge I \lhd J\}$$

Definition 3.3: (Healthy Clock) Clock C is said to be healthy if $\triangle(C) \geqslant u$.

Remarks Here the definition of healthy clock is a little of difference from one of He's healthy clock in [6]. He defines a clock as healthy if $\triangle(C) > 0$ holds.

Our healthy clock depends the unit u. We take the **Clock**.a (G147) as a counterexample. If we choose the unit u as the second and the minute, then the clock **Clock**.a(G147) is healthy. But, if we choose the u as one hour, then this clock is not healthy, since its $\triangle(C) = 19$ minute, which is less than, not greater than or equal to one hour.

Definition 3.4: (Metric of Clocks)

Let $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ be clocks with $D_C = D_E$ and $u_C = u_E$. We define the metric of clocks C and E as to be

$$\rho(C, D) = \sup_{d \in D_C} \{|\beta_C(\lambda_C(d)) - \beta_E(\lambda_E(d))|\}.$$

Theorem 3.5: The family of all clocks over the same index set D with the metric ρ is a metric space, i.e, ρ has the following properties:

(1) ρ is non-negative: $\rho(C, E) \geqslant 0$;
(2) $\rho(C, D) = 0$ if and only if $C = D$;
(3) $\rho(C, D) = \rho(D, E)$;
(4) $\rho(C, E) \leqslant \rho(C, D) + \rho(D, E)$.

C. Clock Operations

This subsection will introduce some basic operators on clock, inspired originally from He's clock work [6].

Definition 3.6: (Projection)

Let $C =< \mathcal{I}, D, \lambda, \beta, u >$ be a clock and $P(d)$ a predicate over D. Set $E = \{d \mid d \in D \wedge P(d)\}$. Call clock $(\mathcal{I}|_E, E, \lambda|_E, \beta|_E, u)$, denoted as $C^{\downarrow P}$, as a projection of clock C with respect to the predicate P, where $\mathcal{I}|_E = \lambda(E)$, a subset of $\mathcal{I}$, whilst $\lambda|_E$ and $\beta|_E$ are the restriction of λ and β on the subset E.

Remark: We need to show that $(\mathcal{I}|_E, E, \lambda|_E, \beta|_E, u)$ is indeed a clock.

Lemma 3.7: If clock C is healthy, then the projection $C^{\downarrow P}$ is healthy.

Definition 3.8: (Filter)

Let $C =< \mathcal{I}, D, \lambda, \beta, u >$ be a clock and $R(x)$ a predicate over the set $\mathbb{R}^+$. We define $\mathcal{I}_R = \{I \mid I \in \mathcal{I} \wedge \exists x \in \mathbb{R}^+.(R(x) \wedge \beta(I) = x)\}$. Then, clock $< \mathcal{I}_R, \lambda(\mathcal{I}_R), \lambda|_{\beta(\mathcal{I}_R)}, \beta|_{\mathcal{I}_R}, u >$, denoted as $C^{\uparrow R}$, is called the filter of clock C with respect to the predicate R.

Definition 3.9: (Shift)

Let $C =< \mathcal{I}, D, \lambda, \beta, u >$ be a clock and $r \in \mathbb{R}^+$. The notation $C^{\rightarrow r}$ represents the clock defined by $< \mathcal{I}, D, \lambda, \beta_r, u >$ where the chronometric function β_r is defined as $\beta_r(I) = \beta(I) + r, \forall I \in \mathcal{I}$.

Lemma 3.10: Both filter and shift as well as projection operations preserve the healthy property.

For given two clocks $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ with the same chronometric unit, i.e., $u_C = u_E$. Then we define the set of instants as the disjoint union $\mathcal{I}_C \sqcup \mathcal{I}_E = \mathcal{I}_C \times \{1\} \cup \mathcal{I}_E \times \{2\} = \{(I_C, 1) \mid I_C \in \mathcal{I}_C\} \cup \{(I_E, 2) \mid I_E \in \mathcal{I}_E\}$ and the index set as the disjoint union $D_C \sqcup D_E = D_C \times \{1\} \cup D_E \times \{2\}$.

For any $d \in D_C \times \{1\} \cup D_E \times \{2\}$, then there exists $d_C \in D_C$ or $d_E \in D_E$ such that $d = d_C$ or $d = d_E$. We define the index function $\lambda_C \sqcup \lambda_E$ as to be

$$\lambda_C \sqcup \lambda_E(d) = \begin{cases} \lambda_C(d_C), & \text{if } d = d_C \\ \\ \lambda_E(d_E), & \text{if } d = d_E. \end{cases}$$

For any $I \in \mathcal{I}_C \times \{1\} \cup \mathcal{I}_E \times \{2\}$, there exists a $I_C \in \mathcal{I}_C$ or $I_E \in \mathcal{I}_E$ such that $I = (I_C, 1)$ or $I = (I_E, 2)$. Then, we define the chronometric function $\beta_C \sqcup \beta_E$as to be

$$\beta_C \sqcup \beta_E(I) = \begin{cases} \beta_C(I_C), & \text{if } I = (I_C, 1) \\ \\ \beta_E(I_E), & \text{if } I = (I_E, 2). \end{cases}$$

Lemma 3.11: For given clocks $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ with the same chronometric unit, i.e., $u_C = u_E$. Then $< \mathcal{I}_C \sqcup \mathcal{I}_E, D_C \sqcup D_E, \lambda_C \sqcup \lambda_E, \beta_C \sqcup \beta_E, u_C >$ is a clock, denoted by $C \sqcup E$.

Definition 3.12: (Disjoint Union)

We call the clock $C \sqcup E$ as the disjoint union of clocks C and E.

Lemma 3.13: If both clocks C and E are healthy, then the disjoint union $C \sqcup E$ is healthy.

Example We may define the clock of G147, **Clock**(G147) as the disjoint union of **Clock**.a (G147) and **Clock**.d (G147). That is,

Clock(G147)= **Clock**.a (G147)$\sqcup$ **Clock**.d (G147).

D. Clock Relations

We define those relations between hybrid clocks, originally from CCSL of André and Mallet's work [1].

Definition 3.14: (Subclock $\subseteq$)

Let $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ be clocks. If $\mathcal{I}_E \subseteq \mathcal{I}_C, D_E \subseteq D_C, \lambda_E = \lambda_C|_{D_E}, \beta_E = \beta_C|_{\mathcal{I}_E}, u_E = u_C$ then we call E as to be a subclock of C, denoted by $E \subseteq C$.

Both **Clock**.a (G147) and **Clock**.d (G147) are subclocks of **Clock**(G147).

Definition 3.15: ((Strict)Faster $(\prec)$ $\preceq$)

Let $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ be clocks. If $D_C = D_E, u_C = u_E$ and for all $d \in D_C$,

$$\beta_C(\lambda_C(d))(<) \leqslant \beta_E(\lambda_E(d))$$

then we call clock C as to be (strict) faster than clock E, denoted by $C(\prec) \preceq E$.

Example Clock.a (G147) $\prec$ **Clock**.d (G147).

Definition 3.16: (Finer $\sqsupseteq$)

Let $C =< \mathcal{I}_C, D_C, \lambda_C, \beta_C, u_C >$ and $E =< \mathcal{I}_E, D_E, \lambda_E, \beta_E, u_E >$ be clocks. If these relations $\mathcal{I}_C \subseteq \mathcal{I}_E$ $D_C \subseteq D_E$, and $\lambda_C = \lambda_E|_{D_C}$ hold and there exists a non-zero natural number k such that $\beta_C = \frac{1}{k}\beta_E|_{\mathcal{I}_C}, u_C = \frac{1}{k}u_E$, then we call clock E as to be finer than clock C, denoted by $E \sqsupseteq C$.

IV. Satisfaction between STeC Design and Hybrid Clock

This section will introduce the notion of satisfaction between STeC design and hybrid clock.

A STeC design refers to a design of a real-time system specified by STeC language. We use the notation clock.S to denote the clock of STeC design S, where instants of clock.S are spatial-temporal points (l, t) of the location l and the time t for a STeC design S.

Now, we define the clock, Clock.S, of STeC design S by structural induction.

$\mathsf{Clock.Send}^{G \rightharpoonup G'}_{(l,t)}(m, \delta) = \{(l, t), (l, t + \delta)\}$.
$\mathsf{Clock.Get}^{G \leftharpoondown G'}_{(l,t)}(m, \delta) = \{(l, t), (l, t + \delta)\}$.
$\mathsf{Clock.Act}^{G}_{(l,t)}(l', \delta) = \{(l, t), (l', t + \delta)\}$
$\mathsf{Clock.Stat}^{G}_{(l,t)}(\delta) = \{(l, t)(l, t + \delta)\}$
$\mathsf{Clock.Sens}^{G}_{(l,t)}(m, \delta) = \{(l, t), (l, t + \delta)\}$
$\mathsf{Clock.Comp}^{G}_{(l,t)}(m, m', \delta) = \{(l, t), (l, t + \delta)\}$
Clock.P;Q= Clock.P ∪ Clock.Q
Clock.P‖Q=Clock.P ⊔ Clock.Q
$\mathsf{Clock}.P \trianglerighteq (\mathsf{Get}^{G \leftharpoondown G'}_{(l,t)}(m, \delta) \to Q)$
= Clock.P∪$\{(l, t)\}$∪ Clock.Q

From these definition of clock.S, we can choose the unit of clock as the minimum of all durations δ which appear in the design S. For example, for the STeC design S-Sensor of sensor,

$$\mathsf{Idle}^{N}_{(l,t)}(60s); \mathsf{Sens}^{N}_{(l,t+60s)}(m, 1s); \mathsf{Comp}^{N}_{(l,t+61s)}(m, 3s);$$
$$\mathsf{Send}^{N \rightharpoonup NC}_{(l,t+64s)}(x, 2s); \mathsf{Idle}_{(l,t+66s)}(10s); \mathsf{Get}^{NC \leftharpoondown N}_{(l,t+76s)}(y, 2s)$$

We can choose the second as the unit of the Clock.S-Sensor, which is the set: $\{(l, t), (l, t+60s), (l, t+61s), (l, t+64s), (l, t+66s), (l, t+76s), (l, t+78s)\}$.

For the STeC design, S-pn, of power network (previous example), we have the clock as follows.

$\{(l_1, t)_1, (l_1, t + 3ms)_1, (l_2, t)_2, (l_2, t + 3ms)_2, (l_1, t + 5ms)_1, (l_2, t + 5ms)_2, (l_3, t + 5ms)_1, (l_3, t + 7ms)_1, (l_3, t + ms)_2, (l_3, t + 7ms)_2, (l_3, t + 7ms), (l_3, t + 12ms), (l_3, t + 12ms)_1, (l_3, t + 14ms)_1, (l_3, t + 12ms)_2, (l_3, t + 14ms)_2, (l_1, t + 16ms)_1, (l_2, t + 16ms)_2, (l_1, t + 21ms)_1, (l_2, t + 22ms)_2\}$.

For Gaotie G147, we have the STeC design S-G147 as previous and have the Clock.S-G147 as follows.

$\{(Beijing, 14 : 30), (Beijing, 14 : 36), (Tianjin, 15 : 10), (Tianjin, 15 : 12), (Jinan, 16 : 15), (Jinan, 16 : 17), (Zaozhuang, 17 : 07), (Zaozhuang, 17 : 09), (Xuzhou, 17 : 28), (Xuzhou, 17 : 36), (Nanjing, 18 : 51), (Nanjing, 18 : 53), (Suzhou, 19 : 42), (Suzhou, 19 : 43), (Shanghai, 20 : 02), (Shanghai, 20 : 11)\}$

Definition 4.1: A design S for real-time system based on STeC language satisfies a hybrid clock C, denoted as $S \models C$, if clock.S$\subseteq C$.

Example: Both S-Sensor $\models$ Clock.S-Sensor and S-pn $\models$ Clock.S-pn. The STeC design S-G147 of Gaotie G147 satisfies the clock **Clock**(G147). That is, S-G147 $\models$ **Clock**(G147).

Lemma 4.2: Let S be a STeC design for a real-time system and two hybrid clocks C_1 and C_2. If S $\models C_1$ and $C_1 \subseteq C_2$, then S $\models C_2$.

V. Conclusion

In this paper, we restate the language of Spatial-Temporal Consistency STeC to specify the behavior of location and times for real-time systems. Then we focus on the introduction of hybrid clock systems to specify both logical and chronometric time of real-time systems. Some operations such as project, filter and disjoint union, on clocks are introduced. Clock relations such as subclocks and finers are introduced too. We set up a satisfaction between STeC design and hybrid clock. These works give a new approach to investigate the time aspect of real-time systems, in particular, CPS systems. Some case studies, such as sensors, trains and power networks show us the valid of hybrid clock system and harmony between STeC language and hybrid clock sytem. We will consider to establish clock logic systems based on hybrid clock system to address the properties of both logical and chronometric time.

Acknowledgements

This work was partially funded by the INRIA Associated Team DAESD between INRIA and ECNU; by NSFC (No. 61021004 and No. 61370100); by Shanghai Knowledge Service Platform Project (No. ZF1213).

References

[1] C. André, F. Mallet, Clock Constraints in UML/MARTE CCSL, Projet Aoste Rapport de recherche Num-6540, May 2008.
[2] G. Berry, The foundations of esterel. In: Proof, language, and interaction. 2000, 425-454.
[3] F. Boussinot, R. De Simone, The esterel language. Proceedings of the IEEE, 1991, 79(9): 1293-1304.
[4] Yixiang Chen Yixiang Chen, STeC: A Location-Triggered Specification Language For Real-Time Systems,2012 IEEE 15th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing Workshops, PP 1-6, 2012.DOI 10.1109/ISORCW.2012.11
[5] C. Fodge, Logical time in distributed computing systems. Computer, 1991, 24(8): 28-33.
[6] J. He, A Clock-Based Framework for Construction of Hybrid Systems. ICTAC 2013, LNTCS 8048: 22-41, 2013.
[7] Zheng Ji, Huiyong Li, Yixiang Chen, Real-time system simulation and verification approach based on STeC to stateflow transformation system. Application Research of Computer, 31(2):448-453, 2014.
[8] L. Lamport, Time, clocks, and the ordering of events in a distributed system. Communications of the ACM, 1978, 21(7): 558-565
[9] Tianjiao Luan, Yixiang Chen and Jiangtao Wang, The Maude rewriting system of the Specification Language STeC for Real-time systems, The Journal of Computer Engineering, 39(10)57-62(67), 2013.
[10] Frédéric, Mallet, Clock Constraint Specification Language Specifying Clock Constraints with UML/MARTE, Innovations in Systems and Software Engineering 4(3)(2008):309-314, 2008.
[11] Object Management Group: UML Profile for MARTE, beta 1. (August 2007) OMG document number: ptc/07-08-04.
[12] Hengyang Wu, Yixiang Chen, Min Zhang. On denotational semantics of spatial-temporal consistency languageCstec. In: Theoretical Aspects of Software Engineering (TASE), 2013 International Symposium on Theoretical Aspects of Software Engineering, IEEE Computer Society, 2013, 113-120.